

Java Number Problems

1. Check if a number is prime.

Simple Explanation: A prime number is a whole number greater than 1 that cannot be exactly divided by any whole number other than 1 and itself. We check by looping from 2 up to the square root of the number. If the number is divisible by any of these loop numbers, it is not prime.

```
public class PrimeChecker {
    public static boolean isPrime(int n) {
        if (n <= 1) {
            return false; // Numbers less than or equal to 1 are not prime
        }

        // Loop from 2 up to the square root of n
        for (int i = 2; i <= Math.sqrt(n); i++) {
            if (n % i == 0) {
                return false; // Found a divisor
            }
        }
        return true; // No divisors found
    }
}
```

2. Generate Fibonacci series using recursion.

Simple Explanation: The Fibonacci sequence starts with 0 and 1, and every subsequent number is the sum of the two preceding ones (0, 1, 1, 2, 3, 5...). Recursion is used by defining the function to call itself with smaller inputs until it reaches the base cases (0 or 1).

```
public class FibonacciGenerator {
    public static int fibonacci(int n) {
        if (n <= 1) {
            return n; // Base cases: fib(0) = 0, fib(1) = 1
        }

        // Recursive step: fib(n) = fib(n-1) + fib(n-2)
        return fibonacci(n - 1) + fibonacci(n - 2);
    }

    // Example usage loop:
    public static void printSeries(int count) {
        for (int i = 0; i < count; i++) {
            System.out.print(fibonacci(i) + " ");
        }
    }
}
```

3. Check if a number is an Armstrong number.

Simple Explanation: An Armstrong number is one that is equal to the sum of its own digits raised to the power of the number of digits. For example, 153 is an Armstrong number because $1^3 + 5^3 + 3^3 = 1 + 125 + 27 = 153$. We first count the digits and then calculate the sum while extracting each digit one by one.

```
public class ArmstrongChecker {
    public static boolean isArmstrong(int num) {
        int originalNumber = num;
        int sum = 0;
        int power = String.valueOf(num).length(); // Get the number of
digits

        int temp = num;
        while (temp > 0) {
            int digit = temp % 10; // Extract the last digit

            // Sum of digit raised to the power of number of digits
            sum += Math.pow(digit, power);
            temp /= 10;
        }

        return sum == originalNumber;
    }
}
```

4. Reverse a number without converting to string.

Simple Explanation: To reverse a number mathematically, we repeatedly extract the last digit using the modulo operator (% 10). We then add this digit to our reversed number, multiplying the reversed number by 10 first to shift the existing digits to the left.

```
public class NumberReverser {
    public static int reverseNumber(int num) {
        int reversed = 0;
        int originalNumber = num;

        while (num != 0) {
            int digit = num % 10; // Get the last digit

            // Build the reversed number (shift current digits left, add
new digit)
            reversed = reversed * 10 + digit;
            num /= 10; // Remove the last digit
        }
        return reversed;
    }
}
```

5. Count digits in a number.

Simple Explanation: To count the digits, we can repeatedly divide the number by 10. Every division removes the last digit. We keep a counter, incrementing it with each division, until the number becomes zero.

```
public class DigitCounter {
    public static int countDigits(int num) {
        if (num == 0) {
            return 1;
        }

        int count = 0;
        int temp = Math.abs(num); // Use absolute value to handle negatives

        while (temp > 0) {
            temp /= 10; // Remove the last digit
            count++;    // Increment count
        }
        return count;
    }
}
```

6. Find GCD of two numbers (Euclid algorithm).

Simple Explanation: The **Greatest Common Divisor (GCD)** is the largest number that divides both numbers evenly. The Euclidean algorithm solves this recursively: $\text{GCD}(a, b)$ is equal to $\text{GCD}(b, a \% b)$. The process stops when the second number (b) is zero.

```
public class GCDFinder {
    public static int findGCD(int a, int b) {
        if (a < 0 || b < 0) {
            throw new IllegalArgumentException("Numbers must be non-negative.");
        }

        if (b == 0) {
            return a; // Base case: GCD(a, 0) = a
        }

        // Recursive step
        return findGCD(b, a % b);
    }
}
```

7. Check if a number is a power of two.

Simple Explanation: A number is a power of two if and only if it is positive and has exactly one '1' bit in its binary representation. A quick bitwise trick is to check if $(n \& (n - 1))$ is equal to zero. This clears the least significant '1' bit, so if the number had only one '1' bit, the result will be zero.

```
public class PowerOfTwoChecker {
    public static boolean isPowerOfTwo(int n) {
        // Must be positive and the bitwise AND of n and (n-1) must be
        // zero
        return (n > 0) && ((n & (n - 1)) == 0);
    }
}
```

8. Compute sum of digits until a single digit remains.

Simple Explanation: This process is called finding the **digital root**. We repeatedly calculate the sum of the digits of the number. If the sum is greater than 9, we repeat the process using the sum as the new number, continuing until the result is a single digit (0-9).

```
public class DigitalRootFinder {
    public static int findDigitalRoot(int n) {
        if (n < 0) {
            n = Math.abs(n);
        }

        while (n > 9) {
            int sum = 0;
            int temp = n;

            // Calculate sum of digits
            while (temp > 0) {
                sum += temp % 10;
                temp /= 10;
            }
            n = sum; // Use the sum as the new number
        }
        return n;
    }
}
```

9. Convert decimal to binary.

Simple Explanation: To convert decimal to binary, we repeatedly divide the number by 2 and keep track of the remainder. The remainders, read from the bottom up (last remainder to first), form the binary equivalent.

```
public class DecimalToBinaryConverter {
    public static String convert(int decimal) {
        if (decimal == 0) {
```

```
        return "0";
    }

    StringBuilder binary = new StringBuilder();
    int temp = decimal;

    while (temp > 0) {
        int remainder = temp % 2; // Remainder is the next binary
digit        binary.insert(0, remainder); // Insert at the beginning
        temp /= 2;
    }
    return binary.toString();
}
}
```

10. Convert binary to decimal.

Simple Explanation: To convert binary to decimal, we start from the rightmost digit and move left. We multiply each digit by 2^n , where 'n' is the position starting from 0. The sum of these products gives the decimal number.

```
public class BinaryToDecimalConverter {
    public static int convert(String binary) {
        if (binary == null || binary.isEmpty()) {
            return 0;
        }

        int decimal = 0;
        int power = 0; // Represents  $2^0$ ,  $2^1$ ,  $2^2$ , etc.

        // Loop backward through the binary string
        for (int i = binary.length() - 1; i >= 0; i--) {
            if (binary.charAt(i) == '1') {
                // Add  $2^{\text{power}}$  to the decimal sum
                decimal += Math.pow(2, power);
            }
            power++; // Increase power for the next digit
        }
        return decimal;
    }
}
```